

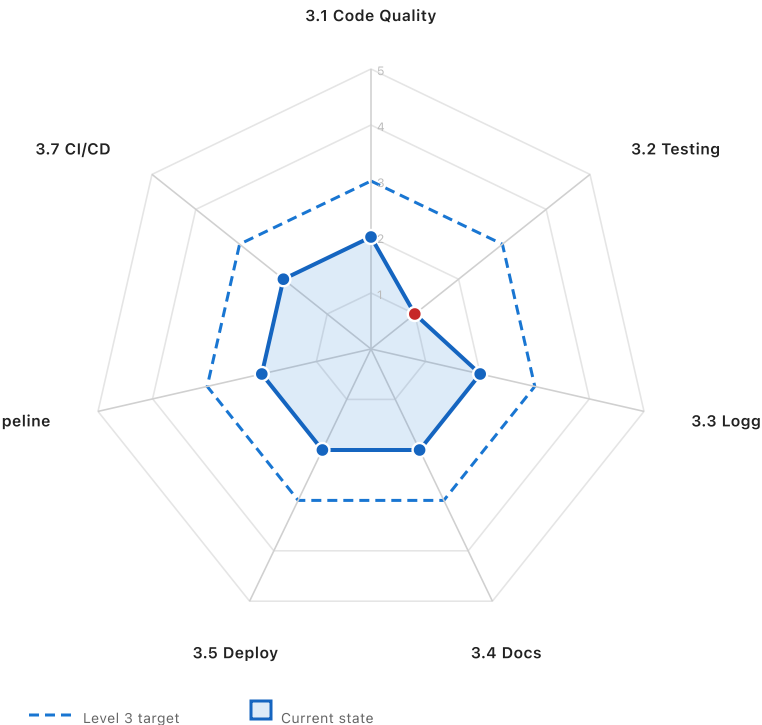
Software Maturity Assessment — kpolimis.github.io

Date: 2026-08-02 · **Framework:** Atlas Analytics Internal Framework v1.0 · **Assessor:** Claude Sonnet 4.6 (automated, evidence-based) · **Tier:** Small / scripts-only · **Tier justification:** Quarto static blog with Python data-viz notebooks and one data-fetch script; Docker and full CI/CD are not required at this tier; `requirements.txt` + `venv` is the standard.

EXECUTIVE SUMMARY

kpolimis.github.io is a Quarto static blog producing World Cup and EPL data-visualization posts via Python notebooks and a data-fetch script (`fetch_wc_data.py`). Six of seven maturity dimensions sit at Level 2 (Documented): practices are written down in `SMALL_PROJECT_STANDARDS.md` and applied by convention, but no tooling enforces them on commit or merge. Testing is the single outlier at Level 1, with zero test files in the repo. The highest-leverage remediation is a four-item, 80-minute sprint (fix the log format, add a `make all` pipeline target, add three missing docstrings, document the footy bootstrap) that moves four dimensions to Level 3 with no new infrastructure. **Theme: Documented but not yet Automated.**

SCORECARD & RADAR



Dimension	Score	Progress
3.1 Code Quality	2/5	
3.2 Testing	1/5	
3.3 Logging	2/5	
3.4 Documentation	2/5	
3.5 Deployment	2/5	
3.6 Pipeline	2/5	
3.7 CI/CD	2/5	

Bars scaled to 5. Dashed ring = Level 3 target. Documented Ad Hoc

Level	Name	Meaning
1	Ad Hoc	Absent; individual habit only.
2	Documented	Written down, not enforced.
3	Automated	Enforced by tooling; violations caught.
4	Monitored	Automated + tracked over time.
5	Optimized	Automated, monitored, gates decisions.

DIMENSION FINDINGS

3.1 Code Quality & Style — 2/5 (Documented)

Evidence: Type hints present on all public signatures in `fetch_wc_data.py` and `.qmd` helpers. Named module-level constants throughout (`TOP_NATIONS` , `TOP_CLUBS` , `MIN_SEGMENT`). No `pyproject.toml` , no Ruff config, no pre-commit hooks — style conventions live in `SMALL_PROJECT_STANDARDS.md` but nothing checks them.

Gap to 3/5: Add `pyproject.toml` with `[tool.ruff]` section; run `ruff check . --fix` once to auto-remediate. Pre-commit hook optional at Small tier.

3.2 Testing & Validation — 1/5 (Ad Hoc)

Evidence: No `tests/` directory, no `pytest.ini` , zero test files anywhere in the repo. Data validation is ad hoc: `pd.to_numeric(errors="coerce")` silently coerces bad values; `os.path.exists()` in notebooks aborts on missing files but does not validate schema. Pure, deterministic functions exist and are trivially testable: `_strip_rank_prefix` , `build_league_stats` , `build_epl_club_stats` .

Gap to 2/5: Create `tests/test_fetch.py` with 5 pytest tests covering the three functions above (no I/O, no mocking needed). Run `pytest` manually before commits.

3.3 Observability & Logging — 2/5 (Documented)

Evidence: `fetch_wc_data.py` now uses `logging.getLogger(__name__)` with `basicConfig` in `main()` — upgraded from 15+ `print()` calls this session. Format is `%(levelname)s %(message)s` , missing the required pipe-delimited standard: `%(asctime)s | %(levelname)-8s | %(name)s | %(message)s` . Notebook cells produce no structured logging (acceptable for a blog at Small tier).

Gap to 3/5: Change the `basicConfig` format string in `fetch_wc_data.py:297` to the pipe-delimited standard. Single-line change.

3.4 Documentation — 2/5 (Documented)

Evidence: `README.md` covers setup and deployment. All six public functions in `fetch_wc_data.py` have Google-style docstrings with `Args:` and `Returns:` blocks (added this session). Three `.qmd` helper functions — `_colour` , `_logo` , `league_colour` — have no docstrings despite being called repeatedly across notebooks. No docstring-coverage enforcement.

Gap to 3/5: Add one-line docstrings to the three undocumented `.qmd` helpers. Add a `pydocstyle` Ruff rule once `pyproject.toml` exists.

3.5 Deployment & Environment — 2/5 (Documented) (Small-tier; Docker N/A)

Evidence: `requirements.txt` created this session with pinned `altair==6.2.2` , `numpy==2.3.5` , `pandas==2.3.3` , `scipy==1.16.3` . `Makefile` created with 9 targets. **Reproducibility gap:** `footy` is a local editable install from `~/repos/football/footy` and is not installable from `requirements.txt` . A fresh clone silently degrades to `HAS_FOOTY = False` . `soccerdata` is also unpinned.

Gap to 3/5: Add a `bootstrap` Makefile target running `pip install -e ~/repos/football/footy` ; document it in `README.md` . Pin `soccerdata` after confirming working version.

3.6 Data Pipeline / MLOps — 2/5 (Documented)

Evidence: Three-stage pipeline: fetch (`fetch_wc_data.py` → 4 CSVs) → transform (inline `.qmd` setup cells) → present (Altair charts). Stage 1 is scripted via `make data` and idempotent. No schema validation at any stage boundary — `pd.read_csv()` loads without column-type or nullability checks; a column rename in FBref would silently corrupt charts. No `make all` target wiring the full pipeline.

Gap to 3/5: Add `make all: data render-wc-leagues render-wc-epl` (one line). Add Pandera column/dtype checks in `.qmd` setup cells to catch FBref schema drift before charts render silently broken.

3.7 CI/CD & Automation — 2/5 (Documented) (Small-tier; full CI not required)

Evidence: `.github/workflows/publish.yml` runs on every push to `main` : renders the Quarto site and deploys to `gh-pages` . No lint, type-check, or test gate before the render step. No branch protection requiring CI to pass before merge. A syntax error in `fetch_wc_data.py` would not block a push.

Gap to 3/5: Add a `make lint` step to `publish.yml` before the render step (3 lines, once Ruff config exists from the 3.1 work). Not strictly required at Small tier but zero additional cost given the existing workflow.

PRIORITIZED ROADMAP

P	Dimension	Action	Effort	Score Δ
P1	3.3 Logging (2→3)	Fix <code>basicConfig</code> format to pipe-delimited standard in <code>fetch_wc_data.py:297</code>	15 min	+1
P2	3.6 Pipeline (2→3)	Add <code>all: data render-wc-leagues render-wc-epl</code> target to <code>Makefile</code>	15 min	+1
P3	3.4 Docs (2→3)	One-line docstrings on <code>_colour</code> , <code>_logo</code> , <code>league_colour</code> in <code>.qmd</code> setup cells	20 min	+1
P4	3.5 Deploy (2→3)	Add <code>make bootstrap</code> target + README note; pin <code>soccerdata</code> version	30 min	+1
P5	3.1 Quality (2→3)	Add <code>pyproject.toml</code> with <code>[tool.ruff]</code> ; run <code>ruff check . --fix</code>	1 hr	+1
P6	3.7 CI/CD (2→3)	Add <code>make lint</code> step to <code>publish.yml</code> before render (<i>after P5</i>)	30 min	+1
P7	3.2 Testing (1→2)	Create <code>tests/test_fetch.py</code> with 5 pytest tests for pure functions	2 hr	+1
P8	3.6 Pipeline (full 3)	Add Pandera schema check in <code>.qmd</code> setup cells for all 4 loaded CSVs	2 hr	—

P1–P4 form a single 80-minute sprint with no new infrastructure. P5 unlocks P6. P7 is the most important gap by risk but the most sustained effort — block a separate 2-hour session.

APPENDIX — RAW EVIDENCE

Area	Detail
CI config	<code>.github/workflows/publish.yml</code> — deploy-only; renders Quarto and pushes to <code>gh-pages</code> ; no lint/test/type-check gate
Logging	<code>fetch_wc_data.py</code> — <code>getLogger(__name__)</code> present; format <code>%(levelname)s %(message)s</code> (off-standard; missing timestamp, level padding, module name)
Docstrings	<code>fetch_wc_data.py</code> — 6/6 public functions documented; <code>.qmd</code> — 3 helpers undocumented (<code>_colour</code> , <code>_logo</code> , <code>league_colour</code>)
Dependencies	<code>requirements.txt</code> pinned: altair 6.2.2, numpy 2.3.5, pandas 2.3.3, scipy 1.16.3; <code>soccerdata</code> unpinned; <code>footy</code> local editable (not installable)
Tests	No <code>tests/</code> directory; no <code>pytest.ini</code> ; 0 test files
Ruff/pyproject	No <code>pyproject.toml</code> ; no Ruff config; no pre-commit hooks
Data validation	<code>pd.to_numeric(errors="coerce")</code> only — silent; no Pandera schema checks; no column-presence assertions at CSV load

Commits (this session)	5 commits: untrack cache artifacts, add Makefile+requirements.txt, logging+docstrings in fetch script, EPL post fixes, leagues post fixes
------------------------	---

Methodology: ~/repos/standards-playbook/software-maturity-assessment-methodology.md · Scale: 1 (Ad Hoc) → 5 (Optimized) · Tier: Small / scripts-only · Assessor: Claude Sonnet 4.6